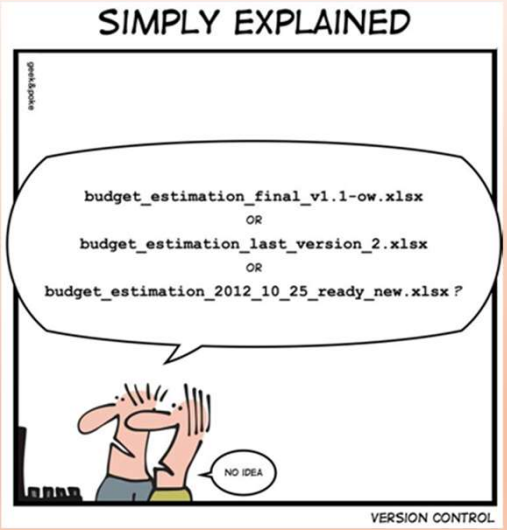
Gestão de Projecto de Software

Licenciatura em Engenharia Informática

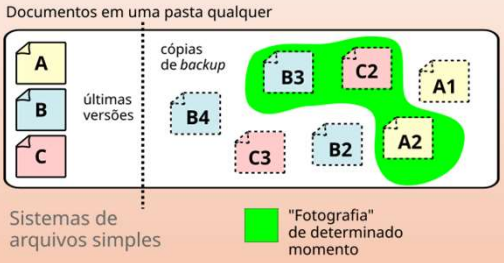
2024/2025

Software de controlo de versões

Controlo de Versão Local .....



- Cópia de ficheiros em pastas
- "alguma" anarquia .... levava rapidamente a alguma desorganização.



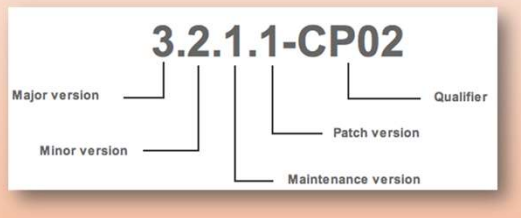
Fonte: https://pt.wikipedia.org/

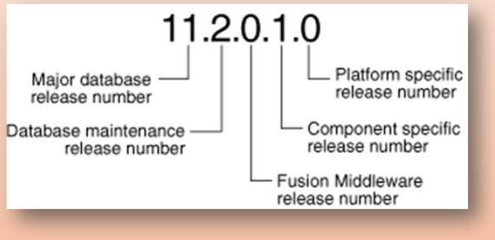
- Nos primórdios (década de 80) surgiu o RCS (Revision Control System)
  - Apenas trabalhada com ficheiros individuais e não com um projecto inteiro
  - Sintaxe complicada

Controlo de Versões de Software

- Vantagens:
  - Controlo do histórico
  - Trabalho em equipa
  - Colocar "marcos" de identificação de versões estáveis
  - Ramificação do projecto
  - Rastreabilidade
  - Segurança
    - Backup
    - Infecção por ´código malicioso (permissões de alteração dos ficheiros)

Codificação





Não há uma norma .... tem que estar documentada

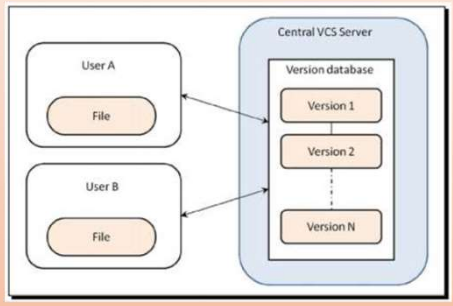
[40] @GPS2324, João Cunha, DEIS-ISEC

Configuration Management

- Configuration Management (CM) is the discipline that ensures that the state of the software at any given time is known and reconstructable. This refers both to software under development and released.
- **Version control** (or revision control, or source control) is the management of documents or programs, keeping track of all the changes.
- **Release control** manages the different releases of product lines, having a common set of files, and different build environments.

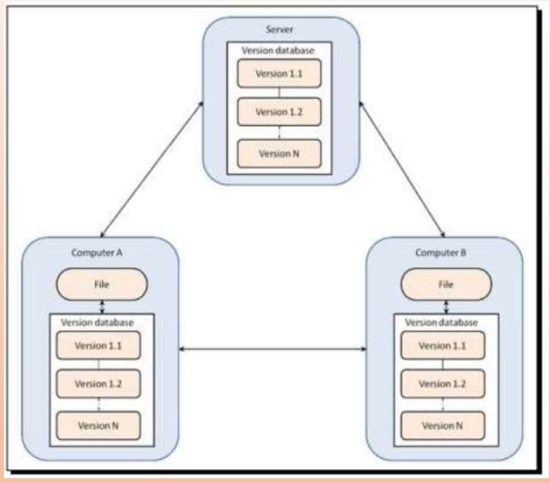
Sistema de Controlo de Versões Centralizado (VCS – version control system)

- As versões do projecto estão armazenadas num único servidor
- Consegue-se ver em que os outros membros da equipa estão a trabalhar
- Qualquer alteração nos ficheiros é compartilhada de forma automática com os outros membros da equipa
- Desvantagens
  - A centralização torna toda a equipa dependente da infra-estrutura de comunicações e do servidor
- Exemplos
  - CVS
  - Subversion
  - ...

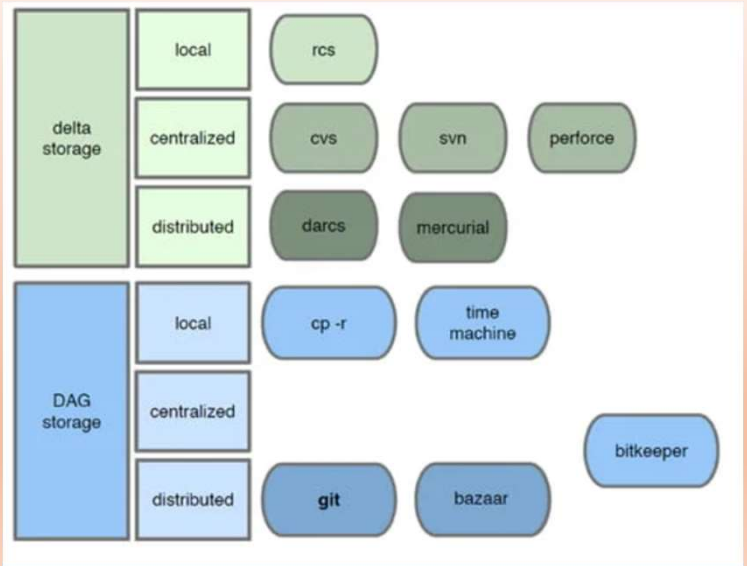


Sistema de controlo de versões distribuído

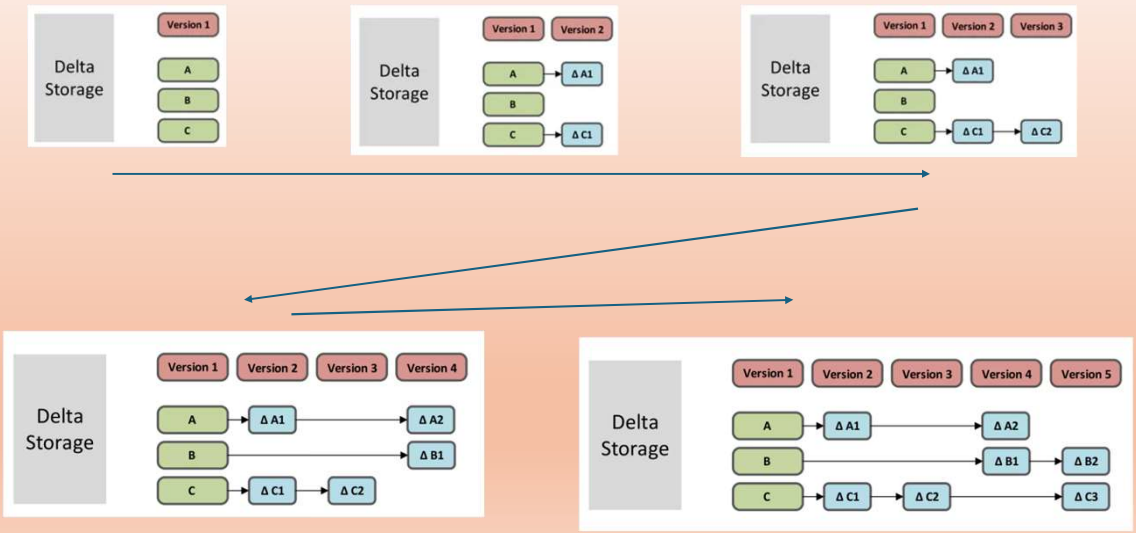
- Sistema distribuído dos ficheiros
  - Em cada computador existe uma cópia integral do projecto
  - Não existe obrigatoriedade de comunicação para se proceder a alterações na máquina local
- Sempre que necessário o utilizador sincroniza as alterações com o servidor, ficando assim disponíveis para os outros membros da equipa.
- Não existe só uma cópia do projecto, há várias e actualizadas.
- Exemplos
  - Git
  - Mercurial
  - Bazar
  - ...



Classificação de algumas soluções..



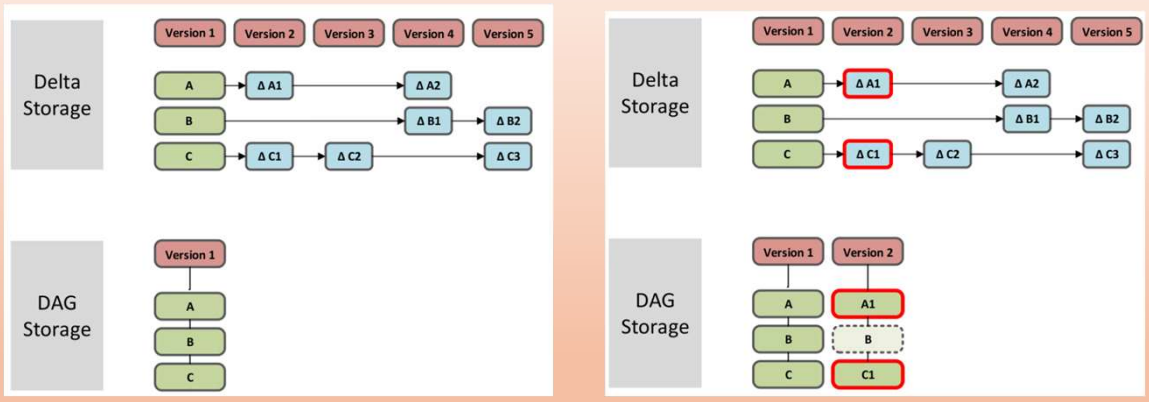
Delta Storage (1)



47

47

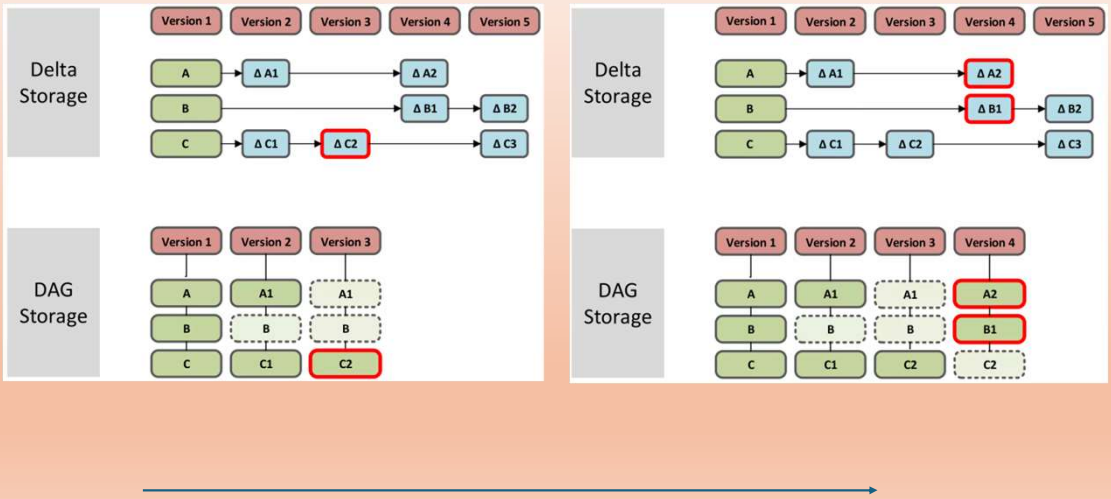
Delta Storage vs DAG Storage (1)



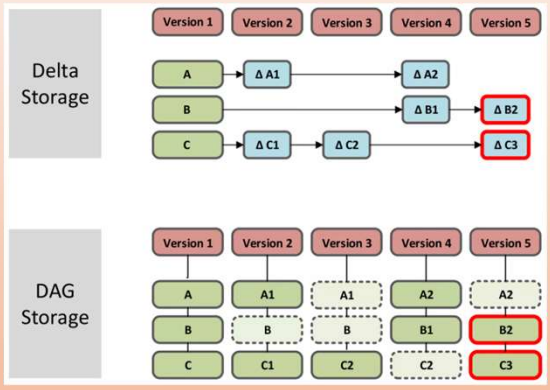
48

48

Delta Storage vs DAG Storage (2)



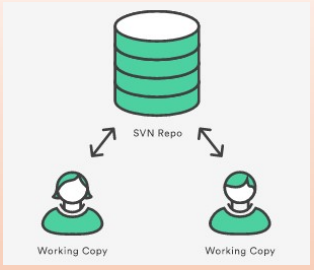
Delta Storage vs DAG Storage (3)



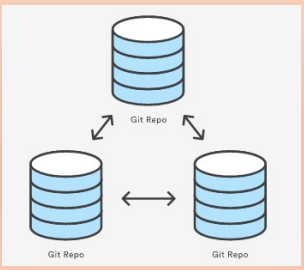
- "file diferences"
- "snapshots"

SVN vs Git

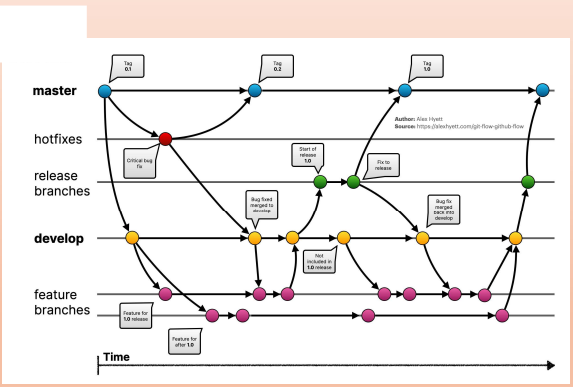
- Apache Subversion (SVN)
  - Open-source project, founded in 2000 by CollabNet. Released in 2004
  - Centralized
  - Delta storage



- Git
  - Created by Linus Torvalds in 2005 for development of the Linux kernel, as open-source
  - Distributed
  - DAG storage







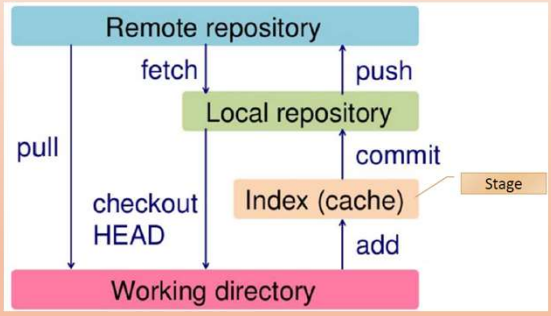
Resumo das principais características do GIT

- Distribuído
- Permite a rastreabilidade do processo
- Facilidade em criar/juntar ramificações
  - branches
  - Merge
- Eficiente e "rápido"
  - Mesmo em repositórios grandes
  - Economia de espaço e tempo

GIT

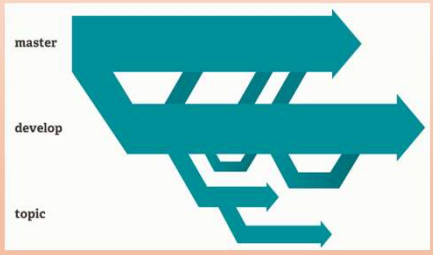
- Linus Torvalds
- 2005

  - "Ciclo de vida"



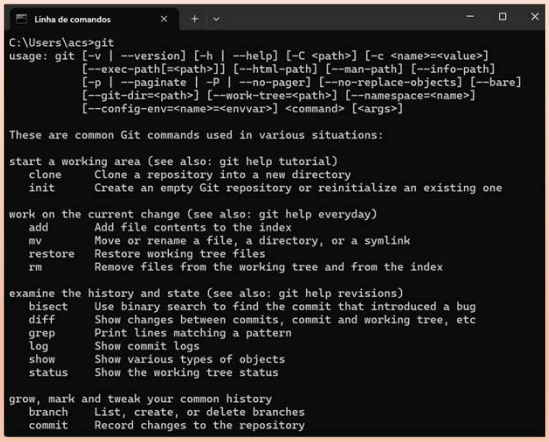
https://git-scm.com/about





GIT

Linha de comando



Interface gráfica / desktop



Comandos básicos do GIT – (linha comando)

“Setup a repositor”

- git init
  - inicializar um novo repositório Git
- git clone [URL]
  - Clonar um repositório remoto para a máquina local.
- git config
  - Figurar o repositório

“Save changes in files”

- git add [arquivo]
  - Adicionar arquivos ao staging area (área de preparação) para serem incluídos no próximo commit.
  - “.” para afectar todos os ficheiros
- git commit -m "mensagem“
  - Registrar as mudanças no repositório com uma mensagem descritiva.
  - Criar um “snapshot”

Comandos básicos do GIT – (linha comando)

“Inspect the repository”

- git status
  - Mostrar o estado actual do repositório (quais arquivos que foram modificados, quais os que estão prontos para commit, etc.)
  - Clonar um repositório remoto para a máquina local.
- Git log
  - Histórico dos commits snapshots

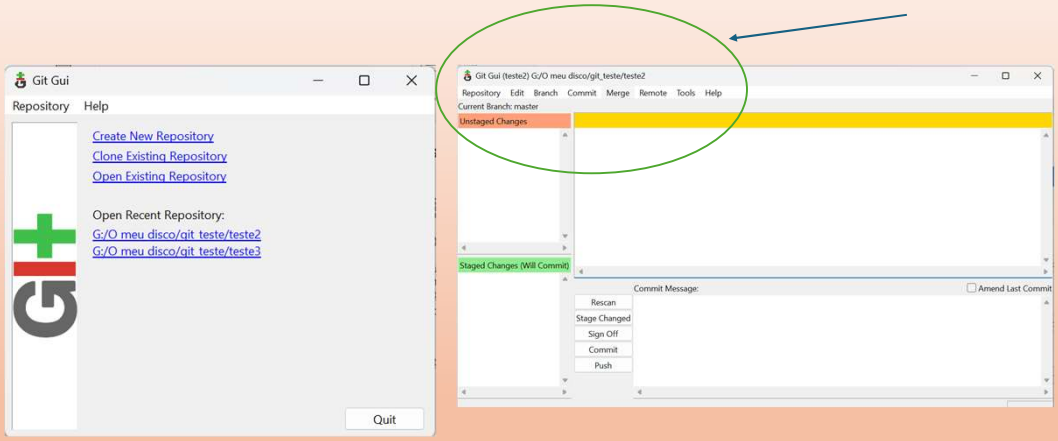
“Undo changes”

- git checkout
  - Mudar para uma versão específica

Comandos básicos do GIT – (linha comando)

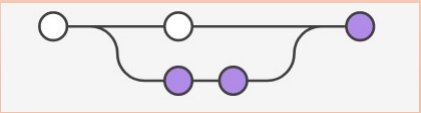
- “Collaborating - Syncing”
  - git remote
    - criar e apagar ligações remotas a outros repositórios
  - git fetch
    - Importar as versões do repositório remoto para o repositório local, não faz “merge”
  - git pull
    - actualizar o repositório local com alterações repositório remoto (faz o “merge”
  - git push:
    - enviar as alterações do repositório local para o repositório remoto

GIT . Interface desktop



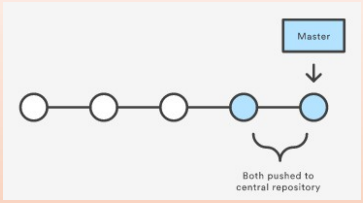
Branches

- **git branch:** create, list, rename, and delete branches
- **git checkout:** updates the files in the working directory to match the version stored in that branch
- **git merge:** take the independent lines of development created by git branch and integrate them into a single branch.

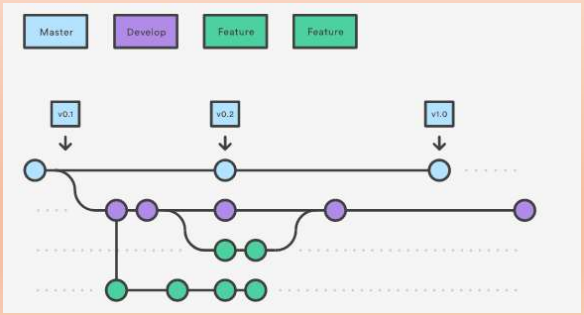


Workflows

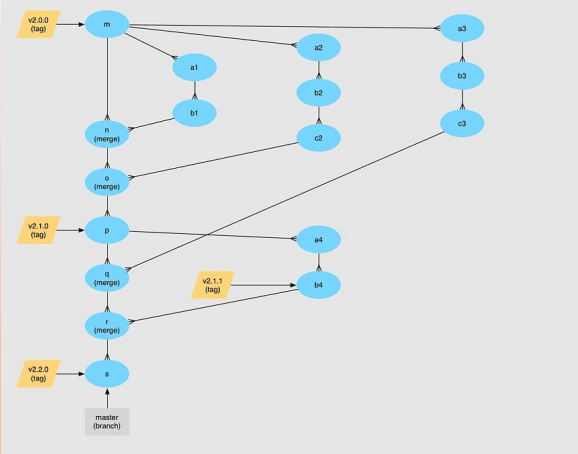
- Centralized workflow



- Feature-based workflow

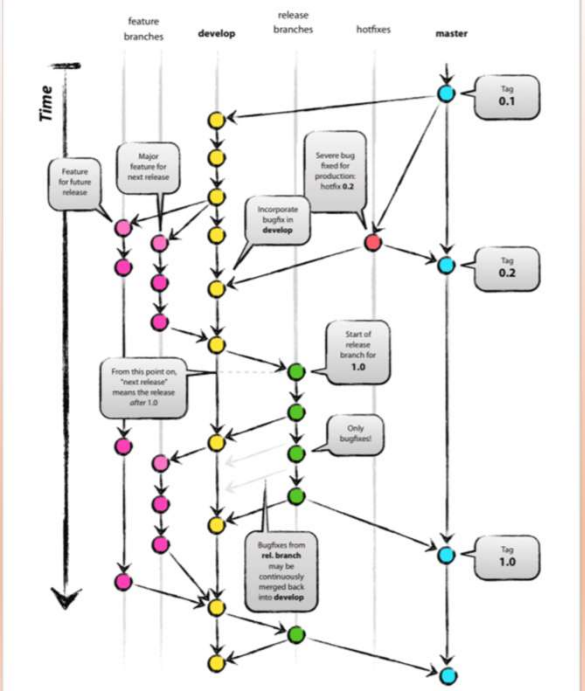


Feature-based



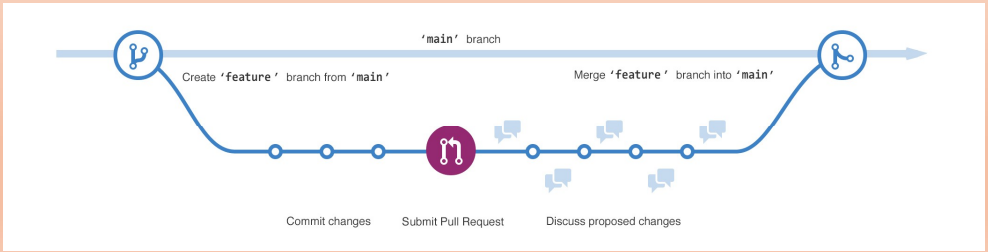
- Desenvolvimento Baseado em Funcionalidades
- Isolamento das Funcionalidades
  - Cada nova funcionalidade ou melhoria é desenvolvida num ramo separado.
  - Evita interferências com o código estável na branch principal (geralmente main ou master)
- Facilita a Colaboração
  - Trabalhar em funcionalidades diferentes em simultâneo
- Integração Contínua
  - As funcionalidades podem ser integradas na branch principal regularmente, após o processo de teste e revisão (muitas vezes facilitado por sistemas de integração contínua (CI))
- Facilita o processo de revisão
  - Antes de fazer o "merge" de uma funcionalidade no branch principal
    - pode-se criar um Pull Request ou Merge Request
      - para revisão de código, assegurando a qualidade e consistência do código.

Gitflow workflow



Pull Request

1. A developer creates the feature in a dedicated branch in the local repo.
2. The developer pushes the branch to a public repository.
3. The developer files a pull request via the repository.
4. The rest of the team reviews the code, discusses it, and alters it.
5. The project maintainer merges the feature into the official repository and closes the pull request.



Further Reading

- GitHub
  - https://guides.github.com/activities/hello-world/
  - https://octoverse.github.com
- Git
  - http://nvie.com/posts/a-successful-git-branching-model/
  - https://www.atlassian.com/git/tutorials
- GitKraken
  - https://www.gitkraken.com/learn-git

- Some slides adapted from A. Stellman & J.Greene, “Applied Software Project Management” and Ian Sommerville, “Software Engineering”

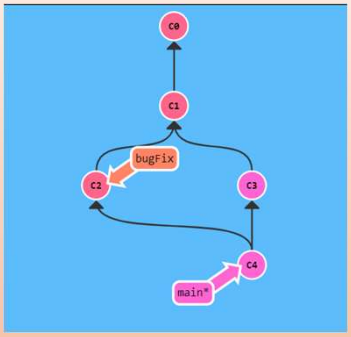
Learn GIT

Oh My GIT!

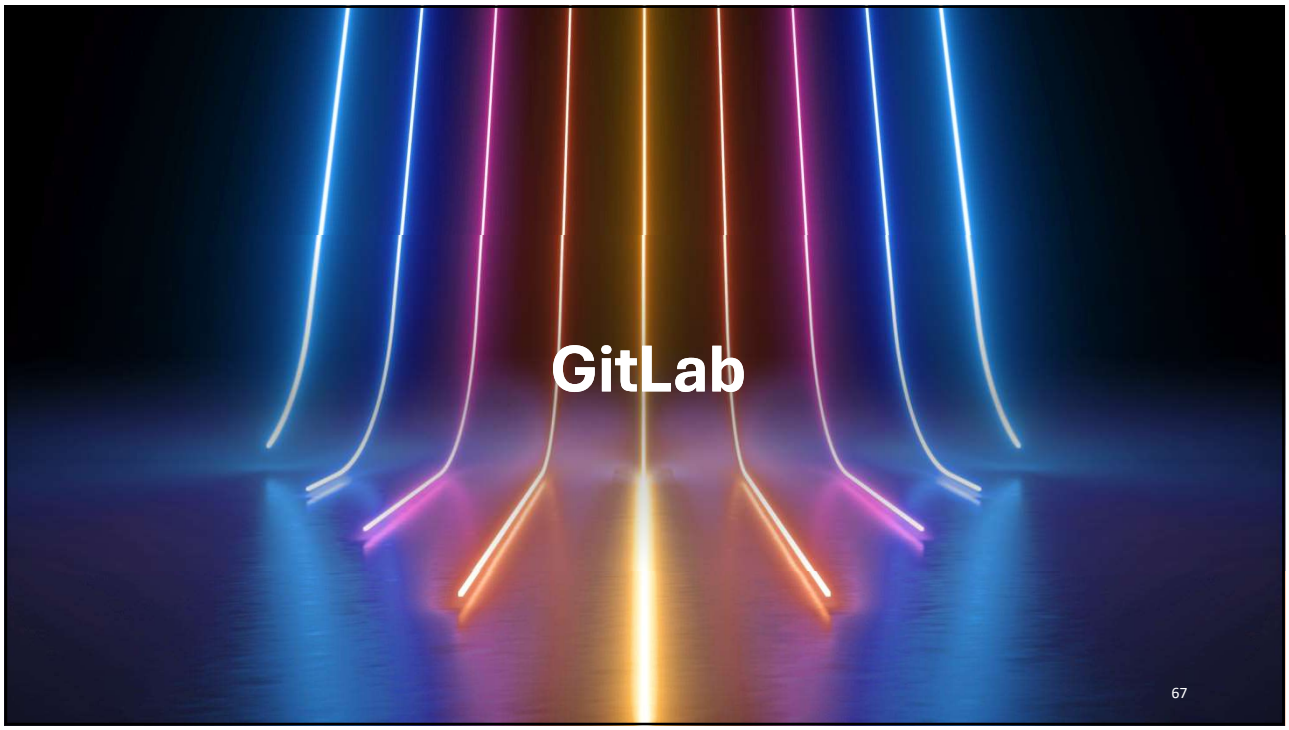


- https://ohmygit.org/

Learn Git Branching



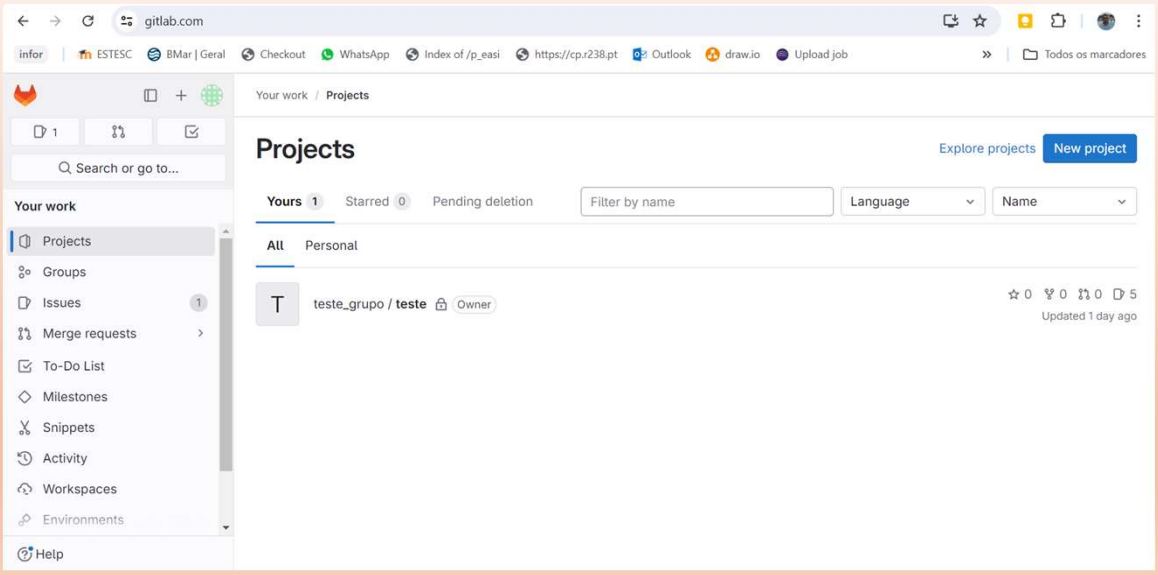
- https://learngitbranching.js.org/



GitLab ?

- ajuda equipas de desenvolvimento de software a trabalharem de forma mais integrada e eficiente
  - permite um ambiente para o desenvolvimento de software,
  - repositórios Git,
  - integração contínua (CI),
  - entrega contínua (CD)
  - monitorização e gestão do projecto
    - planeamento
    - tarefas
    - rastreamento e gestão dos erros
    - ....
  - alternativa ao GitHub, mas com funcionalidades adicionais que tornam o processo de desenvolvimento mais integrado e eficiente.

68



69

69

GitLab - ISSUE

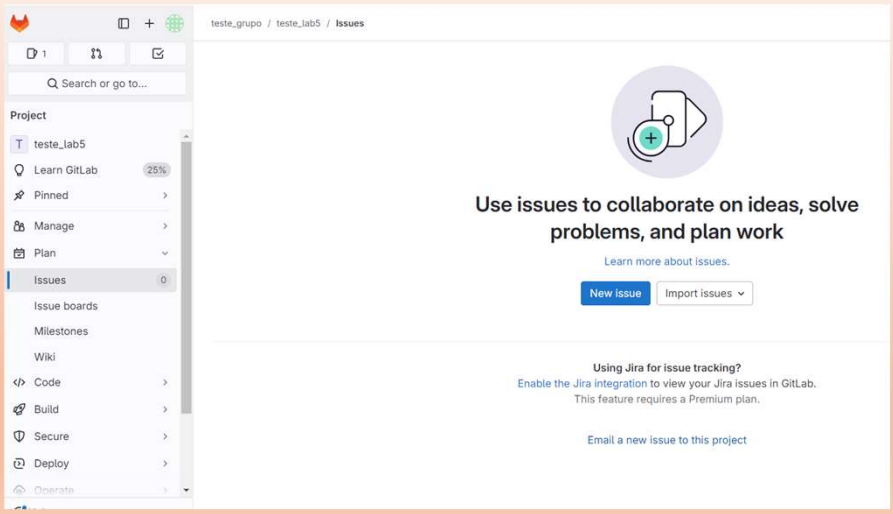
Ferramenta para acompanhar as tarefas, registar bugs, discutir novas funcionalidades, manter um registo da actividade do projecto

- Gerir tarefas
  - Criar e gerir as tarefas, atribuir responsabilidade, definir prazos e acompanhar a evolução
- Rastreamento dos erros
  - Relatar erros encontrados no código, com a possibilidade de inclusão de descrições detalhadas, passos para reproduzir o erro...
- Planeamento de funcionalidades
  - Planear e discutir novas funcionalidades
- Comunicação e colaboração
- Registo do trabalho
  - ´documentar o histórico do desenvolvimento, que facilita processos de revisão e auditoria
- Integração com o código
  - Associação a commits, merge request e branches o que permite o rastreamento da execução de uma tarefa
- Relatório e Dashboards
  - Facilidade de análise da evolução do trabalho

70

70

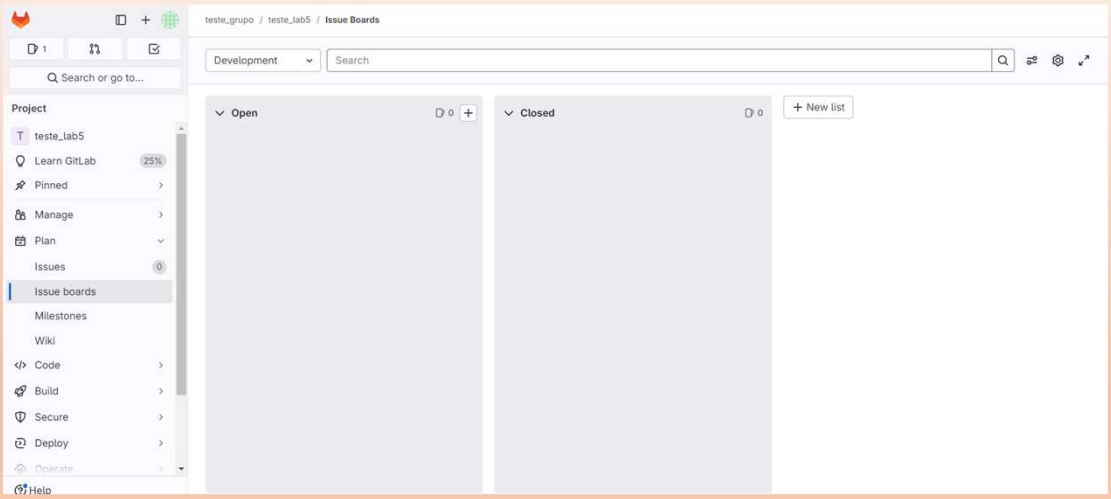
GitLab -> Plan - > Issues



71

71

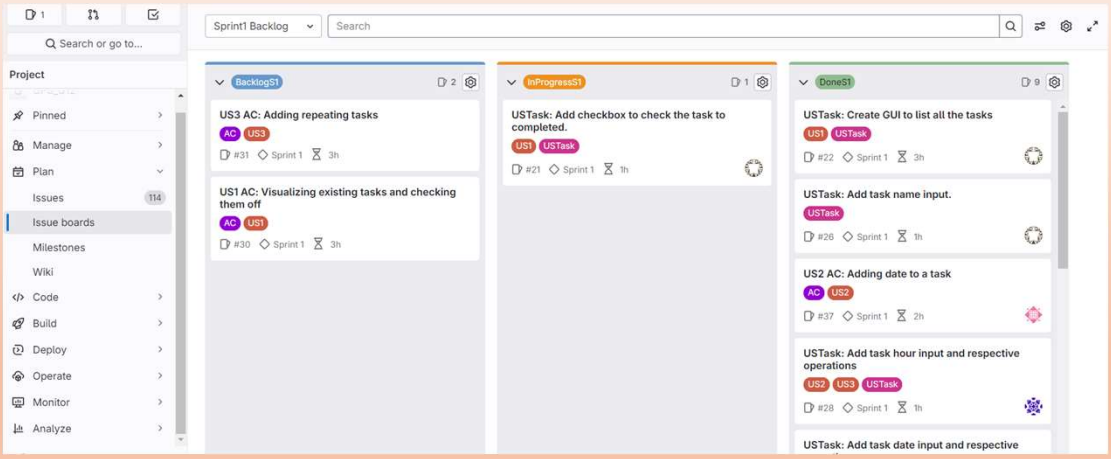
GitLab -> Plan - > Issues Boards



72

72

GitLab -> Plan - > Issues Boards (exemplo prático)

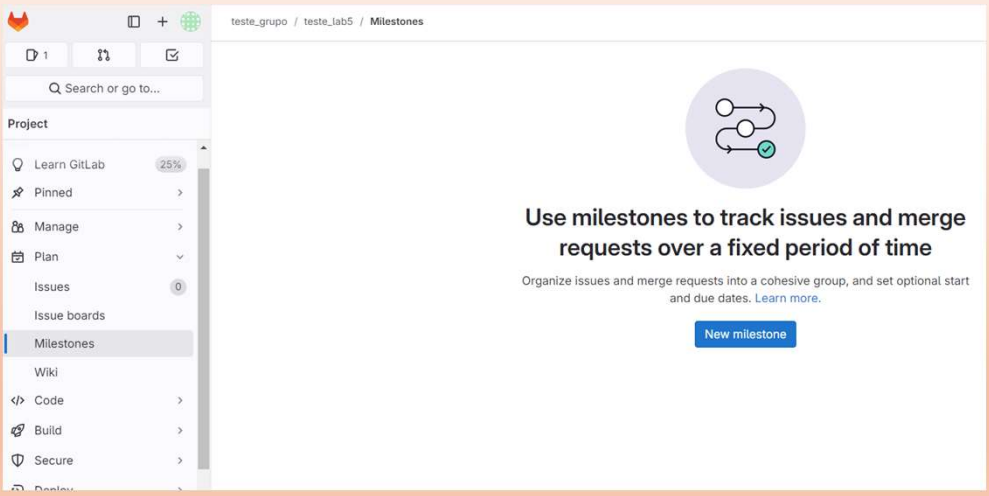


GitLab - milestones

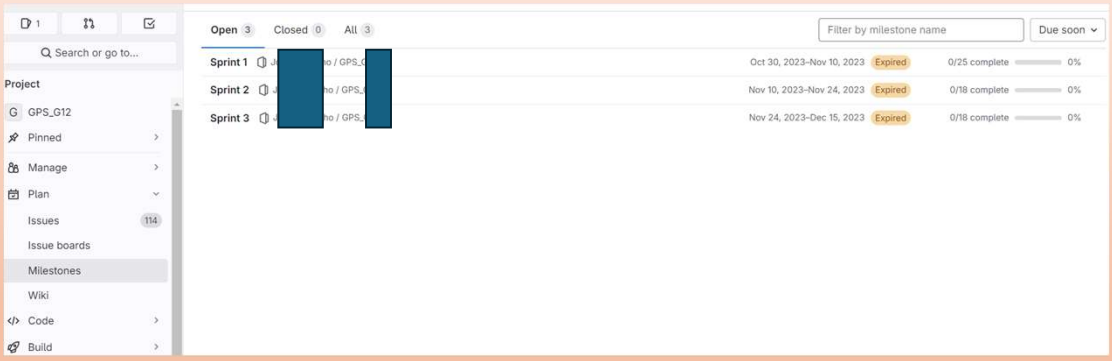
Ferramenta que permite organizar e planear o trabalho em blocos de tempo ou etapas específicas. Permite o acompanhamento da evolução do projectos

- Planeamento de Releases
  - Cada milestones poder, por exemplo, representar uma versão específica do software e organizar e priorizar o trabalho necessário para atingir essa versão.
- Agrupar issues e merge requests
  - Permite agrupar issues e merge request que necessitam de ser concluídos para atingir um objectivo comum
  - Manter o foco da equipa
  - Acompanhar o progresso
- Definição de prazos
- Visibilidade e transparência
  - Todas as partes interessadas podem acompanhar a evolução do projecto
- Coordenação da equipa
  - Garantir o alinhamento e a coordenação entre várias equipas

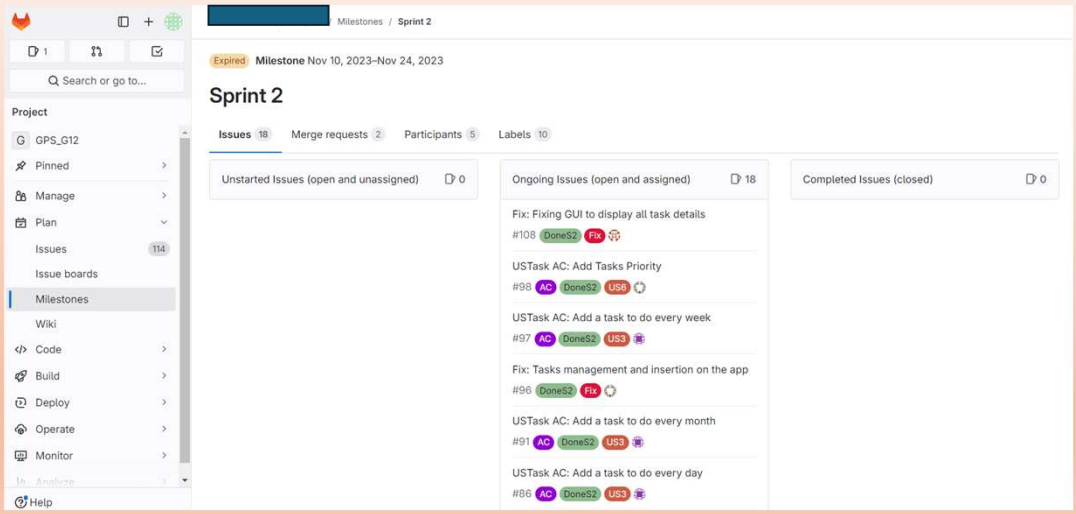
GitLab -> Plan - > Milestones



GitLab -> Plan - > Milestones (exemplo prático)



GitLab -> Plan - > Milestones (exemplo prático 2)

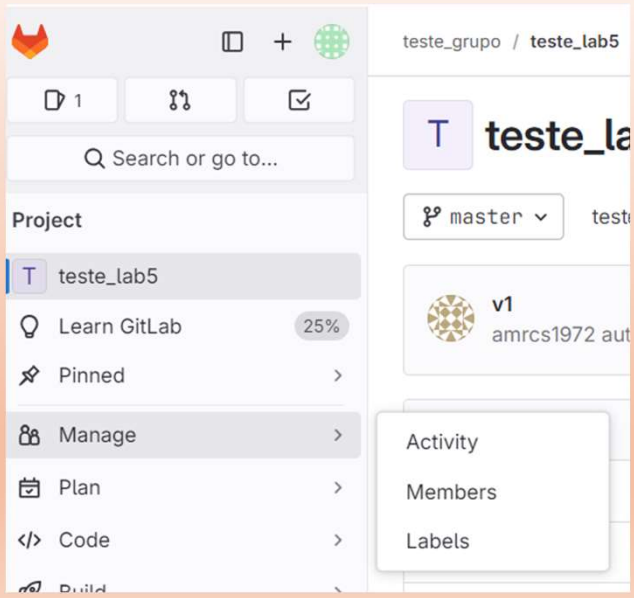


GitLab - Merge Request (MRs)

Ferramenta que permite organizar o desenvolvimento colaborativo de software, nomeadamente o processo de revisão e integração de código.

- Revisão do código
  - Permite a revisão do código pelos outros elementos da equipa antes da integração, o que aumenta a garantia de qualidade e conformidade com os padrões do projecto.
- Discussão e feedback
  - Facilita a comunicação entre os membros da equipa através de comentários e discussões face às propostas de alterações
- Integração Contínua
  - Permite implementar acções de teste e validação automáticas ao código antes de o integrar
- Coordenação da equipa
  - Garantir o alinhamento e a coordenação entre várias equipas
- Histórico e rastreabilidade
  - Registo detalhado faz alterações, discussões e aprovações.
- Controlo de qualidade
  - Permite a configuração de regras de aprovação, por exemplo exigir revisões por determinados elementos da equipa ou por uma equipa antes do processo de integração
- Automatização de revisões
  - Integração com ferramentas de análise de código
- Documentar
  - Permite incluir descrições detalhadas e contextuais das alterações.

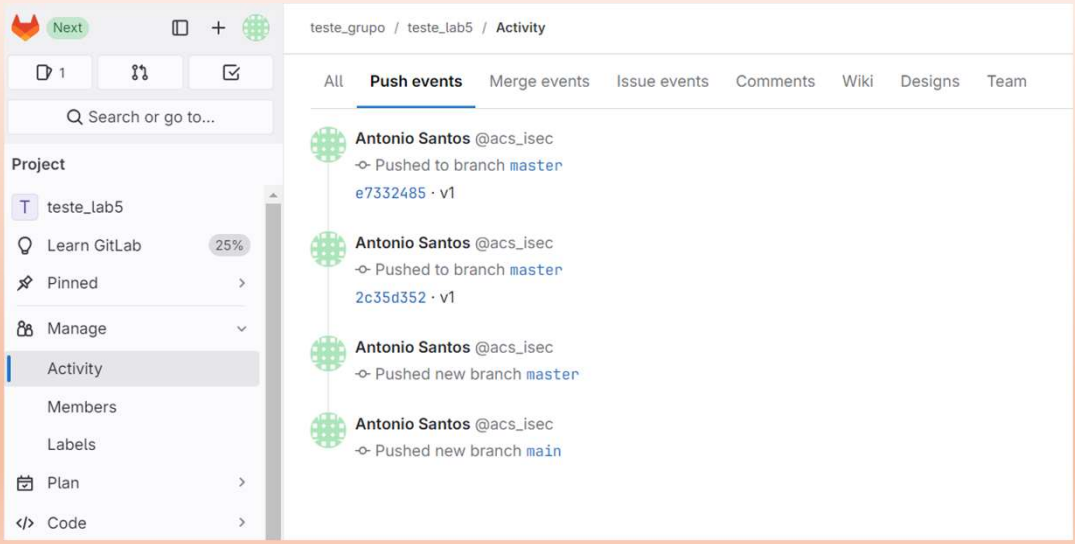
GitLab -> Manage



79

79

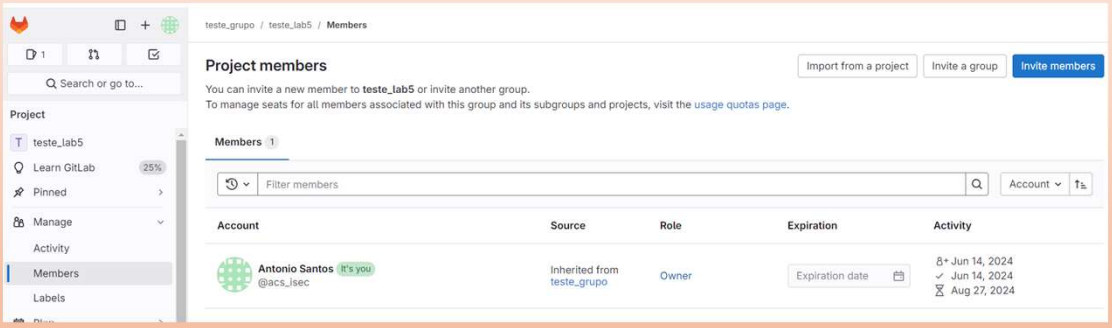
GitLab –> Manage -> Activity



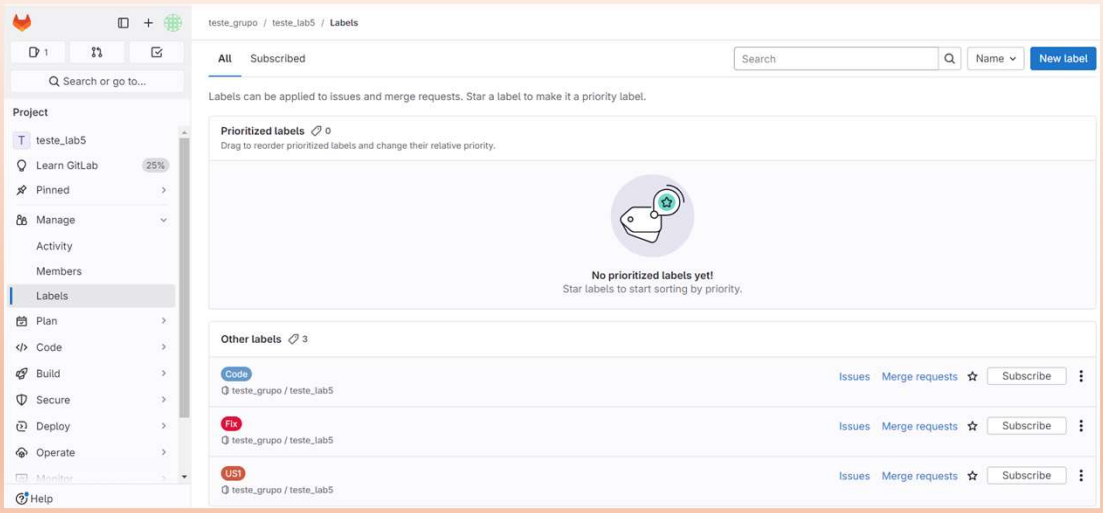
80

80

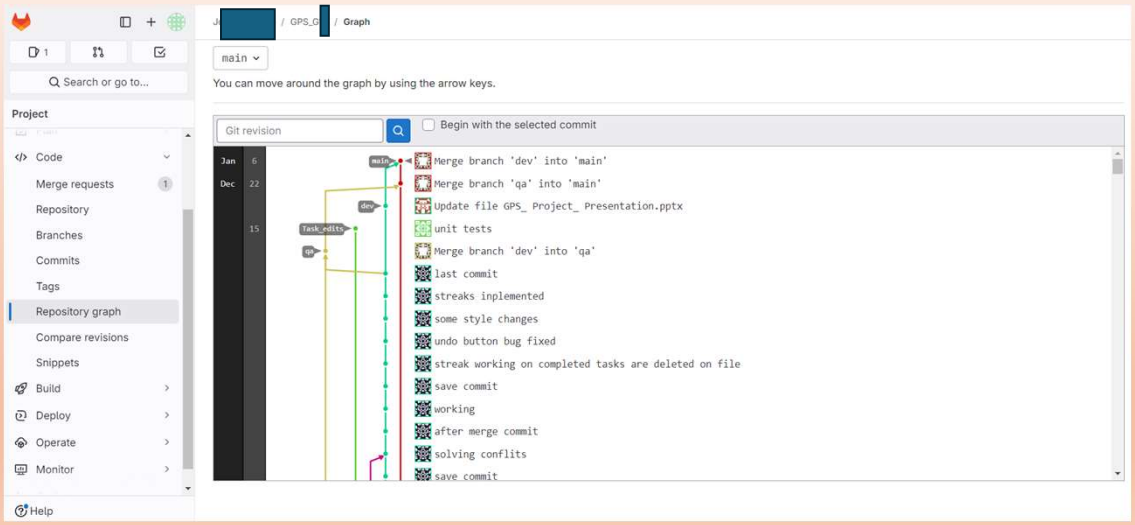
GitLab –> Manage -> Members



GitLab –> Manage -> Members



GitLab –><//> Code



83

83

Modelos de Desenvolvimento de Software

84

84

Modelos de desenvolvimento de Software – 2 caminhos ...

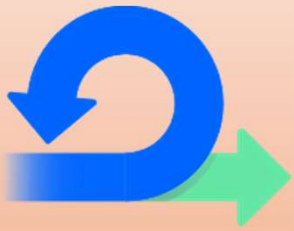
Traditional Process Models

- Waterfall Methodology



Uma cascata ....

Agile Software Development



85

Gestão de Projecto de Software

Licenciatura em Engenharia Informática

2024/2025

Modelos de desenvolvimento de software
Métodos Ágeis
SCRUM

A3